

Memoria del Proyecto: Buscador Web Wikipedia

Sistemas de Almacenamiento y Recuperación de Información · Curso
2025/2026

1. Miembros del Grupo y Contribución

| Miembro | Contribución |
| **Héctor Zamorano García** | Implementación del motor de indexación (`SAR_lib.py`): tokenizador, índice invertido (básico y posicional), algoritmos de cruce por punteros (AND, NOT, MINUS), parser de consultas con operadores booleanos (`parser.py`).
| **Andrés Salas Roger** | Co-desarrollo del módulo de indexación y recuperación. Participación en el diseño de las estructuras de datos del índice invertido, pruebas y validación de las consultas booleanas y posicionales contra los ficheros de referencia del profesor, y contribución a la integración del módulo de búsqueda semántica. |

2. Descripción General del Sistema

El proyecto consiste en un **motor de búsqueda de artículos de Wikipedia** implementado íntegramente en Python. El sistema está compuesto por tres módulos principales:

- `SAR_Indexer.py`: Script de línea de comandos que recorre recursivamente un directorio de ficheros JSON (generados por un crawler de Wikipedia), parsea cada artículo y construye el índice invertido. Soporta las opciones `-P` (índice posicional) y `-S` (índice semántico). El índice resultante se serializa en formato binario mediante `pickle`.
 - `SAR_lib.py` (885 líneas): Biblioteca central que contiene la clase `SAR_Indexer` con toda la lógica de indexación, parsing, tokenización, resolución de consultas booleanas, búsqueda posicional, búsqueda semántica y reranking.
 - `SAR_Searcher.py`: Interfaz de búsqueda que carga un índice previamente construido y permite realizar consultas en modo interactivo (`query:`), por lote (`-L`), individual (`-Q`) o en modo test (`-T`) comparando con ficheros de referencia. También admite búsqueda semántica (`-S`) y reranking semántico (`-R`).
 - `SAR_semantics.py`: Módulo con las clases de modelos de embeddings (Sentence-BERT, BETO, Spacy) y la lógica del KDTree para la búsqueda por similitud vectorial.
-

3. Funcionalidades Implementadas

3.1. Funcionalidades Básicas (Obligatorias)

- **Indexación de artículos**: Recorrido recursivo de directorios, parsing de JSON, detección de duplicados por URL, tokenización con expresión regular (`W+`) y construcción del índice invertido donde cada término mapea a una lista ordenada de `artid`.
- **Consultas booleanas (AND implícito)**: El parser separa los términos de la consulta y aplica intersección AND de forma implícita entre todos ellos. Ejemplo: `futbol español` devuelve artículos que contienen ambos términos.
- **Operador NOT**: Soportado mediante la función `reverse_posting`, que calcula el complemento de una posting list respecto al universo de artículos indexados, utilizando un algoritmo de merge lineal.
- **Operador MINUS (opcional)**: Implementado como diferencia de conjuntos mediante merge de dos listas ordenadas, disponible para futuras extensiones del parser.

3.2. Ampliación 1: Búsqueda Posicional (Comillas)

- **Índice posicional** Cuando se activa la opción `-P`, el índice almacena para cada término una lista de tuplas (`artid`, `[pos1, pos2, ...]`), donde cada posición indica la ubicación exacta del token dentro del campo indexado.
- **Consultas con comillas**: Al introducir una consulta entre comillas (e.g., "selección española de fútbol"), el sistema:
 - 1 Tokeniza la frase interna.
 - 2 Recupera las posting lists posicionales de cada término.
 - 3 Aplica un **algoritmo de intersección posicional pairwise** que verifica que las posiciones de términos consecutivos difieran exactamente en 1, garantizando la adyacencia.
 - 4 Soporta frases de longitud arbitraria (2, 3, 4... palabras).

3.3. Ampliación 2: Búsqueda Semántica y Reranking

- **Indexación semántica** Durante la indexación con `-S`, el texto de cada artículo se segmenta en frases mediante `nlk.sent_tokenize` (idioma español). Cada frase (chunk) se almacena junto con su `artid` correspondiente. Al finalizar, se generan embeddings con el modelo seleccionado y se construye un `KDTree` (distancia euclídea) para búsquedas eficientes por vecinos más cercanos.
 - **Modelos de embeddings disponibles**:
 - SBERT: Sentence-BERT para español (`hiiasid/sentence_similarity_spanish_es`) **modelo por defecto**.
 - BetoCLS / Beto: Modelos basados en BETO (BERT español), usando el token CLS o el promedio de los hidden states.
 - Spacy: Embeddings estáticos de spaCy (`es_core_news_lg`), con opciones para eliminar stopwords y tokens no alfabéticos.
 - **Búsqueda semántica pura (-S umbral)**: Cuando se especifica un umbral, la consulta se convierte en un vector y se buscan los chunks más cercanos en el `KDTree`. El sistema implementa una estrategia de **expansión progresiva** solicita `top_k` resultados iniciales (`MAX_EMBEDDINGS = 200`) y va duplicando la cantidad hasta que la distancia del último resultado supera el umbral o se agotan los embeddings.
 - **Reranking semántico (-R)**: Tras obtener resultados de una búsqueda booleana convencional, los artículos se reordenan según su similitud semántica con la consulta. El sistema recupera chunks del `KDTree` progresivamente hasta cubrir todos los artículos de la búsqueda binaria, y luego los ordena por distancia.
-

4. Decisiones de Implementación y Justificación

4.1. Estructura del Índice Invertido

Se utiliza un diccionario Python (`self.index`) como índice invertido. La decisión de almacenar tuplas (`artid`, `[posiciones]`) en lugar de solo `artid` permite reutilizar la misma estructura tanto para consultas booleanas simples (extrayendo solo los `artid`) como para consultas posicionales, evitando mantener dos índices separados.

4.2. Algoritmos de Cruce por Punteros (sin set)

Para cumplir estrictamente con las normas de la práctica **no se utilizan funciones de conjuntos** (`set()`),

`intersection()`, etc.) en ninguna operación de cruce de posting lists. Todos los algoritmos de intersección (AND), complemento (NOT) y diferencia (MINUS) se implementan con el patrón clásico **merge de dos listas ordenadas con dos punteros**, con complejidad temporal $O(n + m)$.

El algoritmo de intersección posicional extiende esta idea: primero cruza los `artid` comunes y, dentro de cada artículo común, cruza las listas de posiciones verificando la adyacencia (`diff == 1`). Para frases de más de dos términos, se aplica iterativamente de forma pairwise, propagando las posiciones resultantes al siguiente par.

4.3. Segmentación por Frases para Semántica

Se decidió segmentar el texto en frases (`conlltk.sent_tokenize`) en lugar de generar un único embedding por artículo. Un embedding global por artículo mezcla demasiados temas y pierde la capacidad de recuperar artículos relevantes para consultas específicas. La granularidad por frase permite que una consulta semántica como *historia de la selección española* coincida con la frase concreta del artículo que trata ese tema, aunque el artículo completo cubra muchos otros aspectos.

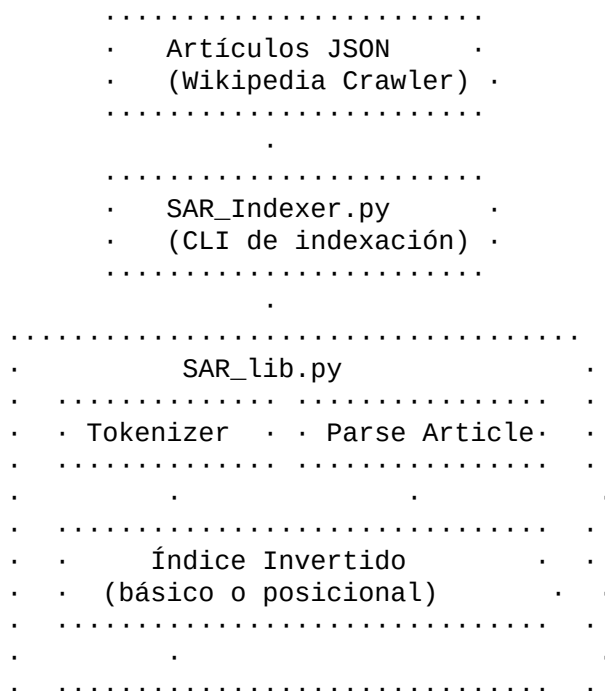
4.4. Estrategia de Expansión Progresiva del KDTree

El parámetro `MAX_EMBEDDINGS` (200 por defecto) limita la cantidad inicial de vecinos solicitados al KDTree. Si la distancia del último resultado está dentro del umbral, se duplica `top_k` y se repite la consulta. Este enfoque **balancea rendimiento y exhaustividad**: evita recuperar todos los embeddings en colecciones grandes, pero garantiza que no se pierdan resultados relevantes.

4.5. Tokenización

Se utiliza una expresión regular sencilla `(\w+)` que divide el texto por cualquier carácter no alfanumérico y convierte todo a minúsculas. Esta decisión prioriza la simplicidad y la velocidad, siendo suficiente para el dominio de artículos de Wikipedia en español.

5. Arquitectura del Sistema



```

. . Resolución de Consultas . .
. . - AND / NOT / Posicional . .
. . - Semántica / Reranking . .
. ....
.
. ....
. SAR_semantics.py .
. ....
. SBERT . . KDTree . .
. BETO . . (sklearn) . .
. Spacy . . . .
. ....
.
. ....
. SAR_Searcher.py .
. (CLI de búsqueda) .
. ....

```

6. Método de Coordinación y Control de Versiones

El trabajo se ha organizado en sesiones conjuntas de programación y revisión. La coordinación se ha llevado a cabo de la siguiente manera:

- **División del trabajo** Se acordó que cada miembro se encargara de funcionalidades específicas, realizando revisiones cruzadas del código implementado por el otro para detectar errores y asegurar la coherencia del estilo.
 - **Validación incremental** Tras implementar cada funcionalidad (indexación básica, AND, NOT, posicionales, semántica), se ejecutaban inmediatamente los tests del profesor con los ficheros de referencia (`test_100.bin`, `test_1000.bin`, `test_100_pos.bin`, `test_1000_pos.bin` y los `.ref` correspondientes) para comprobar que el número de artículos recuperados coincidía exactamente.
 - **Control de versiones:** El proyecto se ha gestionado con Git, manteniendo un repositorio en GitHub (Wikipedia-Search-Engine) con commits descriptivos para cada hito funcional.
-

7. Datasets de Prueba

Se han utilizado los siguientes conjuntos de datos proporcionados por el profesor, de tamaño creciente:

Dataset	Artículos	Uso principal	Uso secundario	Tamaño	Descripción
50_mixed	50	Desarrollo y depuración rápida	100_mixed	100	Validación de consultas básicas y posicionales
500_mixed	500	Test intermedio	1000_mixed	1.000	Validación exhaustiva con referencia
10000_mixed	10.000	Pruebas de rendimiento y escalabilidad			

Ficheros de referencia: `50_mixed_positional.ref`, `100_mixed_positional.ref`, `500_mixed_positional.ref`, `1000_mixed_positional.ref`, `10000_mixed_positional.ref`.

8. Dificultades Encontradas

- **Cruce posicional con más de dos términos** El reto principal fue implementar correctamente la

intersección posicional para frases de longitud arbitraria (3, 4 o más palabras). La solución fue aplicar el cruce pairwise de forma iterativa: se cruza el primer par de términos, se obtienen las posiciones resultantes, y estas se usan como entrada para el cruce con el siguiente término, verificando siempre que la diferencia de posiciones sea exactamente 1.

- **Gestión de la granularidad semántica** Decidir si generar un embedding por artículo o por frase requirió experimentación. Un embedding por artículo producía resultados imprecisos porque mezclaba temas dispares. La segmentación por frases mejoró notablemente la precisión de la búsqueda semántica.
 - **Restauración del KDTree en búsquedas** Al serializar y deserializar el índice con `pickle`, el modelo semántico no se guarda (por su tamaño). El KDTree y los embeddings sí se almacenan en el índice. Al cargar el modelo en tiempo de búsqueda, fue necesario implementar la lógica de `set_kdtree()` y `set_embeddings()` para restaurar el estado del modelo sin recalculr los embeddings.
 - **Compatibilidad de los algoritmos sin `set()`**: Asegurar que todas las operaciones de cruce utilizaran exclusivamente el patrón de merge con punteros requirió disciplina, ya que Python hace trivial el uso de conjuntos. Cada función fue verificada individualmente para garantizar que no se usara ninguna operación de conjuntos.
-

9. Conclusión

El proyecto ha resultado ser una experiencia formativa completa que abarca desde los fundamentos teóricos de los sistemas de recuperación de información hasta técnicas modernas de procesamiento del lenguaje natural:

- Se ha comprendido en profundidad **por qué los motores de búsqueda utilizan índices invertidos** en listas ordenadas y algoritmos de merge con punteros, en lugar de búsquedas por fuerza bruta.
- La implementación de **Índice posicional** ha demostrado cómo una extensión relativamente sencilla de la estructura de datos permite soportar consultas de proximidad y frases exactas.
- La integración de **búsqueda semántica** con Sentence-BERT y KDTree ha permitido explorar el contraste entre la recuperación léxica (basada en coincidencia exacta de términos) y la recuperación semántica (basada en significado), evidenciando la complementariedad de ambos enfoques.
- El **reranking semántico** combina lo mejor de ambos mundos: la precisión de la búsqueda booleana con la relevancia de la similitud vectorial.

El resultado final es un motor de búsqueda funcional, eficiente y extensible, capaz de indexar y buscar en colecciones de hasta 10.000 artículos de Wikipedia con soporte para consultas booleanas, posicionales, semánticas y reranking.